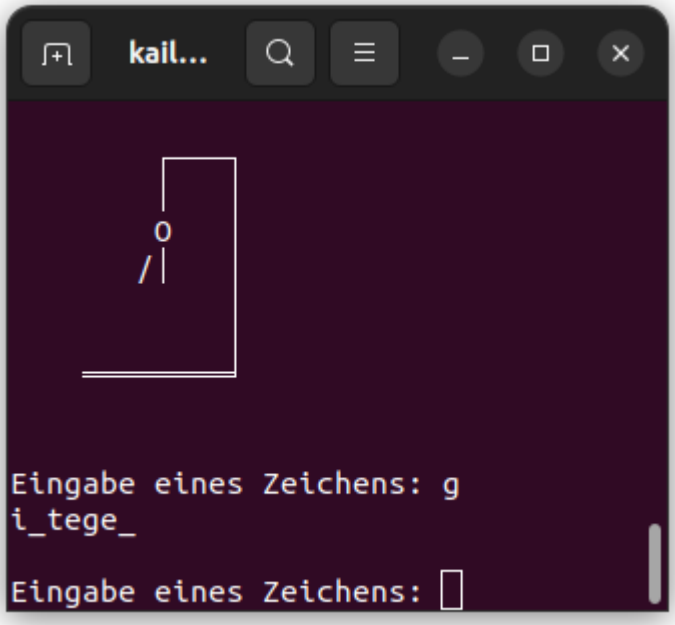
bbs me
Otto-Brenner-Schule

Spieledesign mit MuLE

Spieledesign für den informationstechnischen Unterricht an der exemplarischen Applikation Hangman

Version 0.1, Juni 2025

Autor:
dorau@bbs-me.de



Inhaltsverzeichnis

Inhaltsverzeichnis I
Abkürzungsverzeichnis III
Abbildungsverzeichnis V
1. Einleitung 7
2. Hinweise zum Dokument 8
3. MuLE: Eine Multi-Paradigmensprache für die Bildung 9
3.1 Die MuLE-IDE Eclipse 9
3.2 Das erste MuLE-Programm 11
3.3 Verzeichnisstruktur für die Quellen von Hangman 11
4. Hangman - Das einfache Computerspiel 12
4.1 Spieleprinzip 12
4.2 Das Design von Hangman 12
5. Design und Entwicklung des Computerspiels Hangman 13
5.1 Part 01 – Start mit dem Aufbau eines MuLE-Programms 14
5.2 Part 02 – Interaktion mit einem MuLE-Programm 14
5.2.A Der Datentyp string 15
5.2.B Die Variablen 16
5.2.C Die Eingabefunktion aus der IO-Bibliothek 16
5.2.D Die Ausgabefunktion aus der IO-Bibliothek 16
5.3 Part 03 – Die Spiele-Schleife des Hangman 17
5.3.A Das Loop-Statement 17
5.3.B Die Abbruchbedingung mit if-Statement 17
5.3.C Bestandteile einer Bedingung des if-Statement 18
5.4 Part 04 – Modularisierung des Spiels Hangman 19
5.5 Part 02 – Objektorientierte Programmierung mit Python 20
5.5.A Die Idee der OOP 21
5.5.A.1 OOP - Eigenschaften 21
5.5.A.2 OOP - Methoden 21
5.5.B Die Klasse hangman 23
8. Anlagen 28

8.1 ASCII-Tabelle 28
8.2 Python Referenzkarte 29
8.3 30

Abkürzungsverzeichnis

a.a.O.	am angegebenen Ort
Abs.	Absatz
ASCII	American Standard Code for Information Interchange
Aufl.	Auflage
Bd.	Band
Bsp.	Beispiel
bspw.	beispielsweise
bzw.	beziehungsweise
ders.	Derselbe
DBMS	Database Management System
dies.	dieselbe(n)
Diss.	Dissertation
d. h.	das heißt
Dok.	Dokument
et al.	und andere
etc.	und so weiter
f.	(die) folgende
ff.	(die) folgenden
Fort.	Fortsetzung
ggf.	gegebenenfalls
hj.	halbjährlich
Hrsg.	Herausgeber
hrsg. v.	herausgegeben von
i.d.R.	in der Regel
Jg.	Jahrgang
jhrl.	jährlich
mtl.	monatlich
N.F.	Neue Folge
Nr.	Nummer

o.Ä.	oder Ähnliche(s)
o.ä.	oder ähnlich
o.J.	ohne Jahresangabe
o.O.	ohne Ortsangabe
o.V.	ohne Verfasserangabe
S.	Seite
s.	siehe
Sp.	Spalte
s.o.	siehe oben
sog.	so genannte
SuS	Schülerinnen und Schüler
Tz.	Textziffer
u.	und
u.a.	und andere
u.Ä.	und Ähnliche(s)
u.ä.	und ähnlich
Verf.	Verfasserin / Verfasser
Verl.	Verlag
vgl.	vergleiche
vj.	vierteljährlich
z.B.	zum Beispiel
zit. nach	zitiert nach
TUI	Text User Interface

Abbildungsverzeichnis

Abbildung 1: Eclipse für MuLE 10
Abbildung 2: Das "Hello World" Programm 11
Abbildung 3: Ausschnitt aus dem Eclipse-Spiel Hangman (Console) 13
Abbildung 4: Ausschnitt Project-Explorer der Hangman-Projekte 14
Abbildung 5: Ausschnitt Hello World! auf der Console in Eclipse 14
Abbildung 6: Ein- und Ausgaben und Variablen in MuLE 15
Abbildung 7: vereinfachte Darstellung einer Variablen im Speicher 16
Abbildung 8: Quellcode einer Shopping-Liste 26
Abbildung 9: ASCII-Code Tabelle 28
Abbildung 10: C++ Referenzkarte 29
Abbildung 11: GDB Referenzkarte 31

"Nie haben die Menschen mehr Geist bewiesen, als bei den Spielen,
die sie erfunden haben. Ganz allgemein gesprochen
sind es immer die geistreichsten, die die Spiele erfinden und
die Dümmsten, die diese Spiele am besten spielen."

Gottfried Wilhelm Leibnitz (1646 -1716)

Die Weltgesundheitsorganisation (WHO) hat auf ihrer Jahresversammlung 2019 in Genf endgültig die elfte Version der Internationalen Klassifikation der Krankheiten (ICD-11) verabschiedet und damit auch ***Computerspielesucht als Krankheit*** *anerkannt. Eigentlich handelt es sich dabei um eine weitere Formalie:* ***Gaming Disorder*** *wurde bereits 2018 nach langen Diskussionen in den Katalog der Krankheitsbilder aufgenommen. Damit könne künftig Behandlungskosten von den Krankenkassen übernommen werden.*

http://www.golem.de, 24.05.2019, Peter Steinlechner

1. Einleitung

Computerspiele haben in der Informatik einen festen Stellenwert. Seit den 1980er Jahren haben sie ihren Siegeszug angetreten. Die ersten Spiele waren textbasiert und die Präsentation solcher Spiele war besonders schlicht und einfach gehalten. Bei diesen Spielen stand die Idee – also der Sinn des Spiels – im Vordergrund. Sie hatten eine große Fan-Gemeinde, weil sie modifizierbar waren oder weil Maps oder Levels dazu gebaut werden konnten. Diese Spiele übten deswegen eine große Anziehungskraft auf überwiegend junge Menschen aus und wurden über einen langen Zeitraum gespielt. Mit der Zeit wurden Spiele immer komplexer und es entwickelte sich ein Industriezweig daraus. Spiele wurden vermarktet und sie bilden Lebensgrundlage vieler Spiele-entwickler.

Aus technischer Sicht ist ein Computerspiel hoch komplex und stellt hohe Ansprüche an das Knowhow eines Entwicklers. In einem modernen Spiel sind viele Aspekte der Informatik vereint:

- Hardwarenahe Programmierung einer Grafikengine ggf. in Assembler
- Ein- und Ausgaben zwischen Programm und Spieler (MMI)
- Realistische Simulation von statischen Landschaften in 3D
- Realistische Simulation von bewegten Objekten in 3D (Motion Capturing)
- Plattformunabhängigkeit (Windows/Linux/Unix/Mac-OS/IOS/Android)

Die Entwicklung eines eigenen Computerspiels dient einem tiefgreifenden Verständnis von Programmier- und Designtechniken in der Informatik.

2. Hinweise zum Dokument

In hellblauen Kästen werden Quellcodes dargestellt!

In dunkelblauen Kästen werden die Ausgaben oder Programm-aufrufe dargestellt!

In gelben Kästen werden besondere Informationen darge-stellt!

3. MuLE: Eine Multi-Paradigmensprache für die Bildung

MuLE ist eine Multi-Paradigmen-Sprache, die sowohl anfängerfreundlich als auch gleichzeitig als wirksames Bildungsinstrument verstanden werden kann. Die Sprache unterstützt die drei derzeit wichtigsten Programmierparadigmen: **prozedural**, **objektorientiert** und **funktional**. Um die Anzahl der Konzepte zu reduzieren, mit denen Einsteiger anfänglich konfrontiert werden, bietet MuLE ein gewisses Maß an Abstraktion. Die manuelle Speicherverwaltung ist prinzipiell keine erforderliche Anforderung für Programmieranfänger und ist eher hinderlich für Einsteiger im Anfangsstadium der Programmierausbildung. Daher bietet MuLE implizite Speicherverwaltung und eine sog. Garbage-Collection. Weiterhin bietet es einen minimalen Satz orthogonaler Sprachkonstrukte, die ausdrucksstark sind, um schwer nachvollziehbare Syntax zu vermeiden. MuLE bietet eine klare Syntax, die leicht zu lesen, zu verstehen und zu erlernen ist.

3.1 Die MuLE-IDE Eclipse

Eclipse ist ein open-source Programmierwerkzeug zur Entwicklung von Software verschiedener Art. Ursprünglich wurde Eclipse als integrierte Entwicklungsumgebung (IDE) für die Programmiersprache Java genutzt, aber mittlerweile wird es wegen seiner Erweiterbarkeit auch für viele andere Entwicklungsaufgaben eingesetzt. Für Eclipse gibt es eine Vielzahl sowohl quelloffener als auch kommerzieller Erweiterungen.

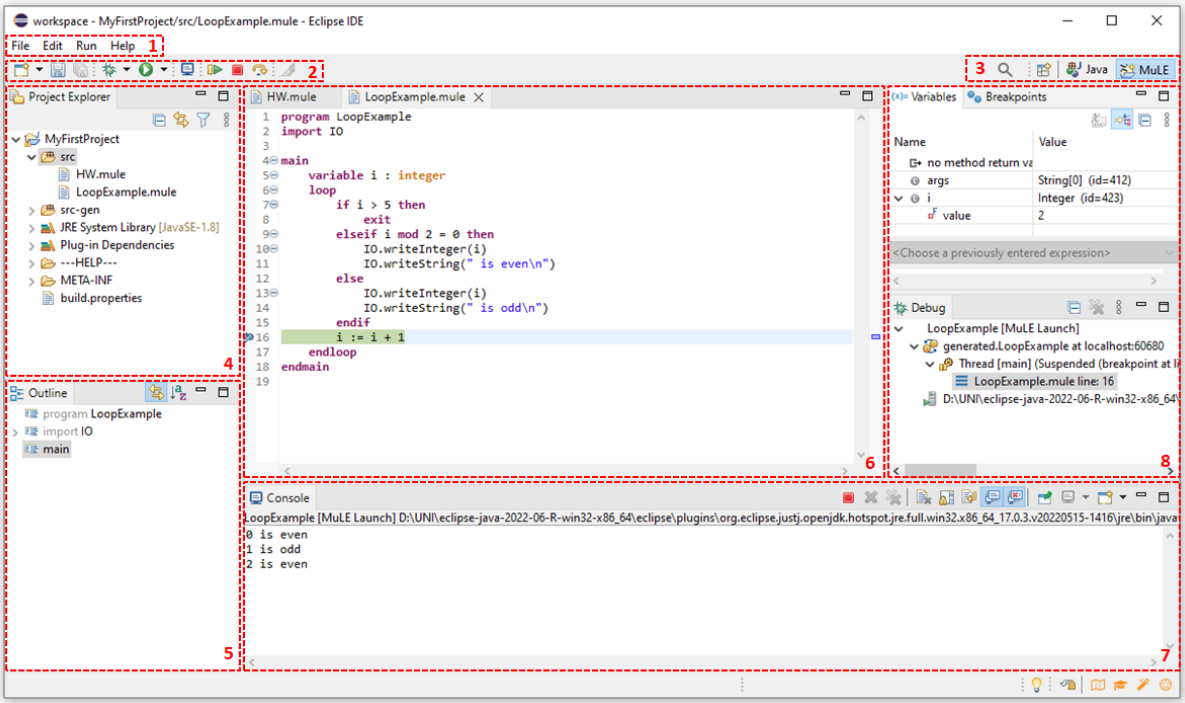


Abbildung 1: Eclipse für MuLE

Nun wird der Blick auf Eclipse gelegt:

1. Dieser Bereich enthält verschiedene Optionsmenüs
2. Dieser Bereich enthält Schaltflächen für den Schnellzugriff auf bestimmte Funktionen
3. Sie können nach Eclipse-spezifischen Befehlen suchen, indem Sie auf die Lupe klicken, und hier Perspektiven verwalten.
4. Dies ist Ihr Projekt-Explorer. Hier können Sie Ihre Projekte durchsuchen, enthaltene Dateien öffnen, Dateien erstellen oder löschen usw.
5. Dies ist die Gliederungsansicht. Sie bietet eine strukturierte Ansicht unseres Programms, was bei großen Programmen mit vielen Typdeklarationen und Operationen hilfreich sein kann.
6. Dies ist Ihr Quellcode-Editor, in dem Sie den Code Ihrer Programme schreiben und bearbeiten.
7. Dies ist die Konsolenansicht. Alle Standard-Ein-/Ausgabefunktionen, z. B. `IO.writeString("Hello, world!")` , werden hier ausgeführt.
8. Und schließlich haben wir einen Bereich mit drei Ansichten, die im Kontext des Debuggens verwendet werden und im Screenshot

dargestellt sind.

3.2 Das erste MuLE-Programm

Die erste Applikation ist das berühmte `Hello World`-Programm, das folgendermaßen aufgebaut sein muss. Es ist in einer Datei `hw.mule` gespeichert.

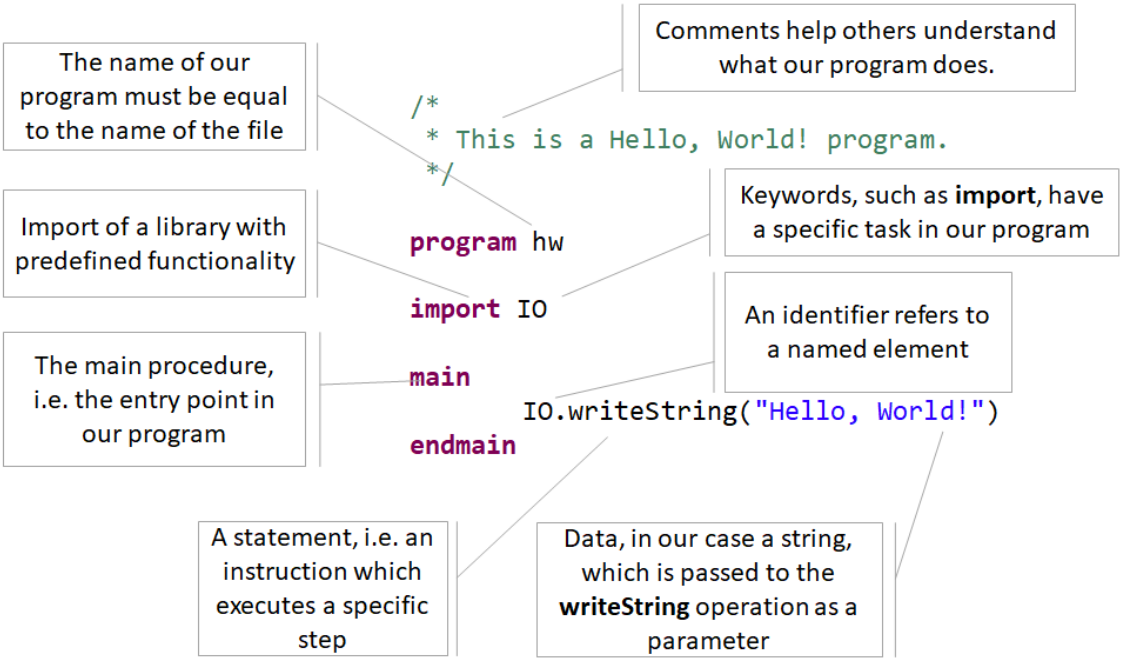


Abbildung 2: Das "Hello World" Programm

Die einzelnen Programmabschnitte sind in der Abbildung beschrieben. Ein Word noch zu den Kommentaren. Bitte mehr kommentieren als zu wenig. Ein Kommentar beginnt mit `/*` und endet mit `*/`, dazwischen können beliebig viele Kommentarzeilen geschrieben werden. Möchte man nur eine Zeile kommentieren, dann kann `//` benutzt werden.

3.3 Verzeichnisstruktur für die Quellen von Hangman

Während der Entwicklung des Computerspiels Hangman ist eine vorgeschriebene Verzeichnisstruktur einzuhalten. In einem beliebigen Verzeichnis werden die Parts, also die Entwicklungsstände, als Verzeichnisse angelegt und dort werden die Quellen gespeichert.

4. Hangman - Das einfache Computerspiel

Tony Augarde, Autor des Werkes The Oxford Guide to Word Games (Oxford University Press) sagt im Kapitel über dieses Spiel: „Die Ursprünge von Galgenmann liegen im Dunkeln, es scheint jedoch in Viktorianischer Zeit entstanden zu sein.“ 1894 ist es in Alice Bertha Gommes Buch Traditional Games unter dem Namen Vögel, Tiere und Fische (engl. Birds, Beasts and Fishes) erwähnt. Die Regeln waren den heutigen Versionen ähnlich.

4.1 Spieleprinzip

Das Programm liest aus einer Datenbank ein zufälliges Wort aus und alle Buchstaben des Wortes werden durch Striche markiert. Der Spieler gibt nun in beliebiger Reihenfolge nacheinander einzelne Buchstaben des Alphabets ein. Das Programm gibt an, wie oft und an welcher Stelle des Lösungswortes der Buchstabe vorkommt. So ergibt sich nach und nach das gesuchte Wort. Kommt ein genannter Buchstabe darin jedoch nicht vor oder hat der Löser gar das falsche Wort geraten, so wird ein Galgen mit einem Gehängten auf dem Bildschirm ausgegeben. Dies geschieht in mehreren Etappen (bei jeder Fehlfrage kommt ein Teilstrich dazu), so dass der Rätsellöser 6 Fehlversuche hat. Hat er dann das Wort noch nicht herausgefunden, so hat er verloren und hängt symbolisch am Galgen.

4.2 Das Design von Hangman

Hangman ist ein sehr einfaches Computerspiel. Es läuft in der Konsole und ist somit auf allen gängigen Hardwareplattformen lauffähig, solange es eine Python-Runtime-Umgebung gibt. Das Spiel benötigt keine grafische Ausgabe und auch keine grafische Bibliothek.

5. Design und Entwicklung des Computerspiels Hangman

Die Spielregeln für Hangman sind nicht besonders schwer. Deshalb ist es recht einfach, die Regeln in ein Computerprogramm umzusetzen. Es sind zwei grundsätzliche Aspekte zu betrachten, zum einen die grafische Repräsentation des Spiels und zum anderen die Umsetzung des Spieleautomaten mit der ganzen Logik für die Spielregeln. Zuerst widmen wir uns der grafischen Ein- und Ausgabe von Hangman. Der Einfachheit halber soll das Spiel in der Eclipse-Konsole laufen. Die Ein- und Ausgaben werden komplett mit den Standard-IO-Funktionen von MuLE getätigt. Die Spielelogik ist eine Schleife mit der entsprechenden Anzahl an Versuchen, die im Hintergrund durchgeführt wird.

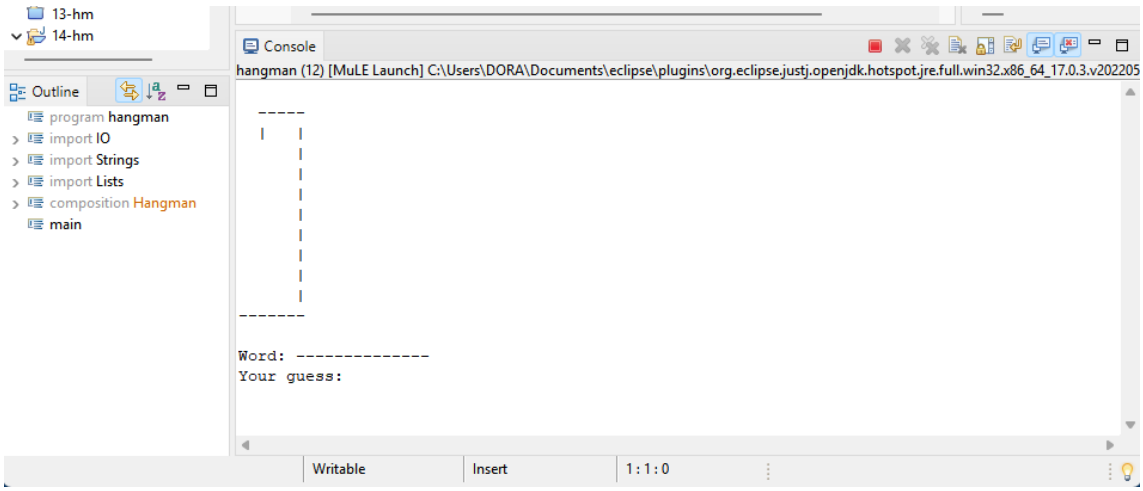


Abbildung 3: Ausschnitt aus dem Eclipse-Spiel Hangman (Console)

Der Projektaufbau in Eclipse ist folgendermaßen gegliedert. Das erste Projekt bekommt den Namen `01-hm`, das zweite Projekt `02-hm` usw. Es wird das erste Projekt angelegt und danach werden alle anderen Projekte aus der Kopie des vorherigen Projekts erstellt. Nach Fertigstellung aller Projekte sieht der Project-Explorer so aus:

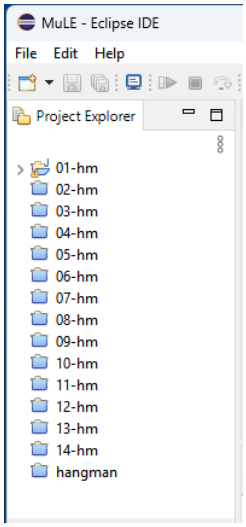


Abbildung 4: Ausschnitt Project-Explorer der Hangman-Projekte

5.1 Part 01 – Start mit dem Aufbau eines MuLE-Programms

Wie in einem vorherigen Abschnitt erwähnt, wird das erste Projekt `01-hw` das Hello-World Programm sein. Die Beschreibung zum Programm befindet sich in 3.2. Die Ausgabe wird so aussehen:

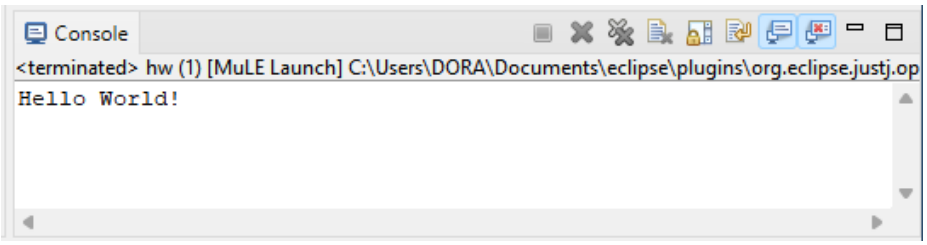


Abbildung 5: Ausschnitt Hello World! auf der Console in Eclipse

5.2 Part 02 – Interaktion mit einem MuLE-Programm

In MuLE werden Ein- und Ausgaben mittels der Bibliothek IO durchgeführt. Folgendes Programm ist gegeben:

```
hangman.mule
1  program hangman
2  import IO
3  import Strings
4
5
6  /*
7   * main hangman
8   *
9   * Theorie
10  * - Variablen
11  * - Datentypen
12  * - Eingaben
13  */
14 main
15   variable str : string
16
17   IO.writeString("Your guess: ")
18   str := IO.readString()
19   IO.writeString(str)
20 endmain
21
```

Abbildung 6: Ein- und Ausgaben und Variablen in MuLE

Zeile	Funktion
2	Import der IO-Bibliothek.
15	Variablendeklaration: Name der Variable ist `str` und der Datentyp ist `string`. Für den String muss die Strings-Bibliothek eingebunden werden.
18	Eingabe eines Strings mittels `IO.readString()` und dessen Speicherung in der Variablen `str`.
17, 19	Ausgabe eines Strings mit `IO.writeString()`.

5.2.A Der Datentyp string

Ein elementarer Datentyp in vielen Programmiersprachen ist der `string`. Die Kernidee jedes Programms besteht daher in der Regel darin, Datenblöcke zu verarbeiten. Um zu wissen, was zu tun ist, hat jede Information einen Datentyp. In diesem Fall ist es der `string` und er stellt Zeichenfolgen dar wie beispielsweise „Hi“ oder „Hallo Welt!“. Die Strings-Bibliothek muss in das Programm eingebunden werden. Bei Strings handelt es sich um einen komplexen Datentyp, der Methoden für die Stringverarbeitung zur Verfügung stellt. Anbei eine kleine Auswahl.

Methode	Funktion

`lengthOf(str : string) : integer`	Gibt die Anzahl der Zeichen zurück
`toUpperCase(str : string) : string`	Wandelt die Zeichen eines String in große Zeichen um
`integerToString(num : integer) : string`	Wandelt eine ganze Zahl in einen String um

5.2.B Die Variablen

Das Konzept der Variablen ist aus dem Mathematikunterricht bekannt. Darin stellen Variablen Platzhalter für Werte dar, z. B. gibt es in `f(x) = mx + t` drei Variablen, wobei `x` gleichzeitig als Parameter fungiert. In den meisten Programmiersprachen fungieren Variablen als benannte Container für Werte, d. h. es können Daten dort abgespeichert werden. Die folgende Abbildung zeigt eine stark vereinfachte Erklärung von Variablen im Speicher eines Programms: es stellt einen Container mit dem Namen `x` und dem Datentyp `integer` dar, in dem ganzzahlige Werte gespeichert werden können. Der aktuell gespeicherte Wert ist `42`.

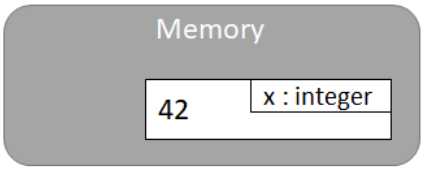


Abbildung 7: vereinfachte Darstellung einer Variablen im Speicher

5.2.C Die Eingabefunktion aus der IO-Bibliothek

Die Funktion `IO.readString()` ließt von der Tastatur einen String ein, den sie als `string`-Wert zurückgibt. Dieser Rückgabewert muss einer Variablen vom Datentyp `string` zugewiesen werden. Die Funktionseingabe wird mit der **Enter**-Taste beendet. Danach steht in der zugewiesenen Variablen die entsprechende Eingabe. Zu beachten ist, dass nur darstellbare Zeichen eingegeben werden können. Sonderzeichen wie die F-Tasten oder die Cursortasten können nicht eingelesen werden. Anzumerken ist, das es noch andere Eingabefunktionen für andere Datentypen gibt, z.B. `IO.readInteger()`.

5.2.D Die Ausgabefunktion aus der IO-Bibliothek

Mit der Funktion `IO.writeString(string)` können Strings ausgegeben

werden. Der Parameter der Funktion kann einen festen String oder eine String-Variable enthalten. Der Verkettungsoperator **&** verkettet Strings miteinander. Es folgen einige Beispiele:

Beispiel	Erklärung
`IO.writeString("hello")`	Ausgabe eines festen Strings
`IO.writeString(str)`	Ausgabe einer String-Variablen
`IO.writeString("hello\n")`	Ausgabe eines festen Strings mit Neuzeile am Ende
`IO.writeString("hello" & str)`	Ausgabe eines festen Strings und einer String-Variablen mit dem Verkettungsoperator. Es können beliebig viele Verkettungen benutzt werden
`IO.writeString(str & "\n")`	Ausgabe einer String-Variablen mit Neuzeile am Ende

5.3 Part 03 – Die Spiele-Schleife des Hangman

Für viele Spiele benötigt man Schleifen und Verzweigungen. Eine Schleife wiederholt ein Programmblock, bis eine Abbruchbedingung erfüllt ist. Die Abbruchbedingung eines Spiels könnte sein, wenn der Spieler eine bestimmte Taste betätigt.

Aufgabe: Schreiben Sie ein Programm in MuLE, das in einer Schleife Eingaben abfragt. Dazu benötigen sie eine String-Variable. Das Programm soll beendet werden, wenn der Benutzer ein **q** eingibt.

Zeitrichtwert: 2 Stunden

5.3.A Das Loop-Statement

Schleifen werden in MuLE mit dem Statement `loop` eingeleitet und mit `endloop` beendet.

```
loop
  // Schleifenblock
endloop
```

Eine Abbruchbedingung fehlt im Moment noch, sodass es sich hierbei um eine Endlosschleife handelt

5.3.B Die Abbruchbedingung mit if-Statement

Die Abbruchbedingung der Schleife muss mit einem if-Statement umgesetzt

werden. Die Abbruchbedingung **muss** immer erfüllt – also wahr - sein, damit der if-Block abgearbeitet wird. Wenn nicht, wird der if-Block nicht abgearbeitet oder der else-Block wird abgearbeitet. Dazu aber später mehr, denn im Moment gibt es keinen else-Block.

```
variable str : string
str := "q"
loop if str = "q" then exit endif
  // Schleifenblock
endloop
```

Wie man sieht, wird die Schleife beendet (`exit`), wenn die Variable `str` das Zeichen **q** enthält. Damit ist die Bedingung erfüllt. Die Abbruchbedingung kann auch anders formatiert werden:

```
variable str : string
str := "q"
loop
  if str = "q" then
    exit
  endif
  // Schleifenblock
endloop
```

5.3.C Bestandteile einer Bedingung des if-Statement

Bedingungen, die beim if-Statement verwendet werden, sind sehr vielseitig. Im oberen Beispiel ist die Bedingung `str ="q"`, und gehört damit zu den Vergleichsoperatoren. Es wird verglichen, ob die Variable `str` das Zeichen **q** beinhaltet. Es gibt noch weitere Vergleichsoperatoren:

Vergleichoperatoren	Funktion
=	gleich, auf jeden Typ anwendbar
/=	ungleich, auf jeden Typ anwendbar
<	kleiner als, anwendbar auf numerische Typen
>	größer als, anwendbar auf numerische Typen
<=	kleiner als oder gleich, anwendbar auf numerische Typen
>=	größer als oder gleich, gilt für numerische Typen

Es gibt aber noch boolsche Operatoren, die meistens genutzt werden, um mehrere Vergleichsoperatoren in einer Bedingung zu verknüpfen. Z.B. x < 5

und y > 10. Der boolsche Operator ist hier **und**.

Boolsche Operatoren	Funktion
`and`	Der Ausdruck ist wahr, wenn alle Vergleiche wahr sind, andernfalls falsch
`or`	Der Ausdruck ist wahr, wenn mindestens ein Vergleich wahr ist, falsch nur, wenn alle Vergleiche falsch sind
`not`	negiert den Ausdruck

Beispiele:

Beispiel	Funktion
`x>0 and x<10`	x muss Werte zwischen 1 und 9 enthalten, dann wahr, sonst falsch
`x=0 or x=10`	x muss entweder den Wert 0 oder 10 enthalten, dann wahr. Wenn keiner der Werte vorhanden ist, falsch
`not (x<5)`	Wenn x kleiner 5, dann wäre der Ausdruck wahr, wird aber negiert, also falsch. Ist x größer oder gleich 5, wäre der Ausdruck falsch, wird aber negiert und ist damit wahr.

5.4 Part 04 – Modularisierung des Spiels Hangman

Um den Programmcode übersichtlicher und wiederverwendbar zu gestalten, können in MuLE Funktionen definiert werden. Funktionen sind Programmblöcke, die beim Aufruf abgearbeitet werden. Sie können beliebig oft aufgerufen werden und sind damit wiederverwendbar. Ein einfaches Beispiel folgend:

Return type of the operation

```
import IO

operation f(parameter x : integer) : integer
    return 2 * x + 1
endoperation

main
    variable x : integer
    x := -2
    loop
        if x > 2 then exit endif
        IO.writeInteger(f(x)) IO.writeLine()
        x := x + 1
    endloop
endmain
```

Return statement

$$f(x) = 2x + 1$$

x	-2	-1	0	1	2
$f(x)$	-3	-1	1	3	5

Fragmente	Funktion
`operation`, `endoperation`	Die Schlüsselwörter leiten eine Funktionsdefinition ein
`f`	Ist der Name der Funktion. Er kann beliebig gewählt werden.

`parameter x : integer`	Funktionen können Parameter übergeben werden (hier eine Variable x vom Typ Integer). Diese Parameter (Variablen) gehen von außen in die Funktion hinein und können in der Funktion genutzt werden. Parameter können beliebig durch Komma getrennt aneinandergereiht werden.
`: integer`	Der Rückgabewert der Funktion ist von Typ Integer
`return 2 * x + 1`	Der Funktionsrumpf beinhaltet den Code, der bei Aufruf abgearbeitet werden soll. Mit dem Schlüssenwort `return` werden Ergebnisse aus der Funktion zurückgeliefert.
`IO.writeInteger(f(x))`	Aufruf der Funktion, die jetzt abgearbeitet wird. Der Parameter x wird der Funktion übergeben. Der Rückgabewert der Funktion (2*x+1) wird über die Ausgabefunktion auf dem Bildschirm ausgegeben.

Aufgabe: Schreiben Sie ein Programm in MuLE, das die Eingabe in einer Schleife von Part 03 in einer Funktion auslagert. In der Funktion soll die Schleife mit der Eingabe durchgeführt werden. Der Rückgabewert ist die entsprechende Eingabe. Im Hauptprogramm soll die Funktion aufgerufen und der Rückgabewert angezeigt werden.

Zeitrichtwert: 2 Stunden

5.5 Part 05 – Objektorientierte Programmierung mit MuLE

Die objektorientierte Programmierung (OOP) ist neben der imperativen und der deklarativen ein weiteres Programmier-Paradigma. Paradigmen beschreiben grundsätzliche Denk- und Arbeitsweisen. In der Informatik wird vor allem der Programmier-Stil verstanden. Insgesamt haben sich schon viele verschiedene Programmier-Paradigmen herauskristallisiert. Praktisch kann man mit jeder Herangehensweise ein Programm für ein spezielles Problem erstellen. Dabei sind der Aufwand und die Qualität des Programms oft sehr unterschiedlich. Ziel ist aber immer ein fehlerfreies, effektives, gut lesbares, redundanzfreies, modulares und nebenwirkungsfreies Programm zu entwickeln. Gute Programmierer wählen immer eine spezielle Programmiersprache, die meist unterschiedlichen Paradigmen zugehören, um ein Problem zu lösen. Bei dem objektorientierten Ansatz geht es darum, Eigenschaften (Attribute) und Funktionen (Methoden), die zu einem Objekt zugehören, gemeinsam zu verwalten und zu programmieren.

5.5.A Die Idee der OOP

In der OOP wird von Objekten gesprochen, um die Materie greifbar zu machen.

- Objekte haben Eigenschaften wie z.B. Größe, Farbe und Alter
- Objekte können Aktionen machen / bzw. mit diesen Objekten gemacht werden wie z.B. knurren, schmusen und schlafen

Bisher wurden in MuLE Daten deklariert, die mit Funktionen verändert wurden. Die Funktionen dienen einer modularen Aufteilung des Programms. Was passiert aber, wenn Daten und Funktionen miteinander verknüpft werden? Dann entsteht daraus der Kerngedanke der OOP. Unspezifische Datenstrukturen wie Variablen, Listen oder Tupel werden in Objekte beschrieben.

Als konkretes Beispiel soll eine **Katze** dienen. Die ist orange, fett und frisst nur Lasagne, falls sie nicht schläft und heißt **Garfield**. Die Eigenschaften sind:

- hat eine Farbe
- hat ein Alter
- hat einen Namen
- hat 4 Beine

Durch einen „Bauplan“ kann nun ein allgemeines Bild einer Katze definiert werden. Das ist eine Katzen-Klasse. Und nun können virtuelle Katzen in beliebiger Anzahl erstellt werden – sprich ganz viele Objekte, die grundlegend gleich nach dem Bauplan aufgebaut sind, aber sich in Ihren Eigenschaften (Farbe, Alter, Name) unterscheiden.

5.5.A.1 OOP - Eigenschaften

Eigenschaften beschreiben die statischen Attribute eines Klasse. Bei der Katze sind es wie oben beschrieben Farbe, Alter, Name und Anzahl der Beine. Meist werden Eigenschaften nur einmal gesetzt und dann selten verändert.

5.5.A.2 OOP - Methoden

Methoden sind im Grunde genommen Funktionen, die ein dynamisches Verhalten einer Klasse beschreiben. Methoden einer Katze wären:

- Fressen
- Schlafen

- Schmusen
- Fauchen
- Spielen

Die Katze kann also, sobald die Methode `fressen()` aufgerufen wird, den Futternapf leeren (oder die Maus verspeisen). Je nach Objekt (nicht jede Katze ist gleich) kann eine Katze spielen, schmusen oder fauchen – muss aber nicht. Prinzipiell wäre es bezüglich der Katzenkomposition möglich.

Composition Katze	**Objekt 1**	**Objekt 2**	**Objekt 3**
Eigenschaften			
Farbe	Orange	Schwarz/Weiß	Braun
Alter	3	10	1
Name	Sammy	Leo	Mieze
Methoden			
Miauen	miauen()		
Schlafen	schlafen()		
Fressen	fressen()		
Schmusen	schmusen()		

- Kompositionen sind allgemeine Beschreibungen von Objekten, also eine Blaupause
- Objekte haben konkrete Werte als Eigenschaften und können alle über definierte Methoden interagieren.

Folgende Begriffe der OOP sind definiert:

- Komposition, also der Bauplan von Objekten einer Komposition (Blaupause)
- Eigenschaften, wie sieht ein Objekt aus?
- Methoden, was kann das Objekt tun?
- Objekte, aus einer Komposition erstellte Instanzen
- Instanz, die Erzeugung eines Objekts (lateinisch: abgeschlossene Einheit)

5.5.B Die Komposition Hangman

Die Hangman-Komposition wird aus Eigenschaften und Methoden definiert. Das folgende Listening zeigt die Hangman-Komposition:

```
hangman.mule
1  program hangman
2  import IO
3  import Strings
4
5
6  /*
7   * class hangman
8   * - class definition
9   * - attributes
10  * - operations
11  * - visibility
12  */
13 type Hangman : composition
14   // private attribute
15   private attribute str : string
16
17   // public operation
18   operation op(parameter str : string) : string
19     loop if str = "q" then exit endif
20       str := IO.readString()
21     endloop
22     return str
23   endoperation
24 endtype
25
26 /*
27  * main hangman
28  *
29  * - initiation of a hangman instance
30  * - call operation of class
31  */
32 main
33   variable str : string
34   variable hm : reference<Hangman>  // reference to hangman object
35
36   hm := reference Hangman{str = ""} // creating hangman object
37
38   IO.writeString("quit, press q and enter: ")
39   str := hm@.op(str)                // call hangman operation op()
40   IO.writeString("program has finished with ")
41   IO.writeString(str)
42 endmain
43
```

Zeile	Funktion
13	Die Kompositionsdefinition startet mit `type name : composition`
15	Die Eigenschaft `str` wird definiert. Im Grunde ist `str` eine Variable mit einem Datentyp. In der Komposition wird eine Eigenschaft mit dem Schlüsselwort `attribute` generiert.
18-23	Die Methode `op` ist definiert. Sie hat die gleiche Syntax wie eine

	Funktion. Es können Parameter und Rückgabewerte existieren. Eingeleitet wird eine Methode mit dem Schlüsselwort `operation`.
34	Es wird eine Referenzvariable für ein Objekt benötigt. Dabei muss die Komposition (hier Hangman) angegeben werden.
36	In dieser Zeile wird ein Objekt der Komposition Hangman erstellt. Das Objekt heißt `hm` und über diesen Namen können Eigenschaften und Methoden angesprochen werden.
39	Die in der Komposition Hangman definierte Methode `op` wird aufgerufen. Die Methode hat ein Parameter `str` und ein Rückgabewert, der in `str` gespeichert wird.

5.5.C Sichtbarkeiten einer Komposition

Eigenschaften und Methoden können in ihrer Sichtbarkeit eingeschränkt werden. Das erhöht die Sicherheit der Ausführung des Programms. Eigenschaften einer Komposition sind mit wenigen Ausnahmen immer sehr eingeschränkt sichtbar. Sie sollen nicht von außen verändert werden können. Das wird mit dem Schlüsselwort `private` erreicht. In der aktuellen Hangman-Komposition ist `str` nur private, also nur innerhalb der Komposition sichtbar. Die Methode dagegen ist `public`, also auch von außen sichtbar und aufrufbar.

Sichtbarkeit	Funktion
`private`	Eigenschaften oder Methoden können nur innerhalt der Komposition verändert oder aufgerufen werden. Außerhalt der Komposition sind sie nicht gültig. Ergibt ein Höchstmaß an Sicherheit.
`protected`	Eigenschaften oder Methoden sind in der eigenen Komposition, aber auch in abgeleiteten Kompositionen sichtbar. In Hangman gibt es aber keine Ableitungen, sodass dieses Schlüsselwort nicht relevant ist.
`ohne Schlüssel-wort`	Eigenschaften und Methoden sind `public`, also von innerhalb und außerhalb der Komposition sichtbar. Das ist speziell bei Eigenschaften unsinnig. Bei Methoden dagegen ist es sinnvoll, weil das Verhalten von Objekten von außen gezielt verändert werden soll.

In unserem Beispiel ist die Eigenschaft `str` nur innerhalb der Komposition veränderbar und die Methode `op` von außen aufrufbar.

Aufgabe: Setzen Sie das Programm wie im Quellcode zu sehen um. Erzeugen Sie nicht nur ein sondern drei Objekte der Komposition Hangman und rufen die Methode `op` jeweils von den drei Objekten auf.

Zeitrichtwert: 4 Stunden

5.6 Part 06 – Eingabe eines Zeichens für Hangman

Das Spiel Hangman benötigt eine Eingabe. Es sollen Zeichen eingegeben werden, um ein Wort zu erraten. Die eingegeben Zeichen sollen auf alle Fälle in große Zeichen umgewandelt werden, um eine einheitliche Darstellung zu gewährleisten.

Aufgabe: Erzeugen Sie eine neue Eigenschaft in der Komposition `private attribute aUsed : list<string>`. Diese Eigenschaft speichert die Zeichen, die schon eingegeben worden sind. Schreiben Sie eine Methode `checkGuess() : string`, in der ein Zeichen eingegeben werden kann. Dieses Zeichen soll in ein großes Zeichen umgewandelt werden. Es soll in der Methode geprüft werden, ob das Zeichen schon in der Liste `aUsed` steht. Wenn ja, soll ein Leerstring zurückgegeben werden. Wenn nein, dann soll das eingegebene Zeichen in die Liste `aUsed` angehängt und dann zurückgegeben werden. Testen Sie die Methode in dem Hauptprogramm.

Zeitrichtwert: 8 Stunden

5.6.A Listen als komplexer Datentyp

In der Programmierung ist eine Liste eine Datenstruktur, die Daten strukturiert speichert. Im Gegensatz zu Kompositionen, die ebenfalls Datenstrukturen sind, können in Listen keine Werte unterschiedlichen Typs gespeichert werden. Der Datentyp muss anfänglich festgelegt werden. Folgende Vorgehensweise ist nötig.

Fragment	Funktion
`import Lists`	Für Listen muss eine Bibliothek eingebunden werden
`variable lst : list<string>`	Zunächst muss eine Variable mit dem Datentyp `list` deklariert werden. Nach den Datentyp `list` muss noch in Dreiecksklammern der Datentyp benannt werden, der in der Liste als Elemente gespeichert werden soll. In diesem Fall sind es String-Elemente.

`lst[0] := "hello"`	Listenelemente werden indiziert. Das geschieht mit den eckigen Klammern und einem Index. In diesem Fall wird in das erste Listenelement der String **hello** geschrieben.
`IO.writeString(lst[0])`	In diesem Fall wird das erste Listenelement gelesen und auf dem Monitor angezeigt.

In der Listenbibliothek existieren eine Menge von Listenmethoden, die auf Listen angewendet werden können. Nachfolgend eine kleine Auswahl.

Methode	Funktion
`isEmpty(l : list<Type>) : boolean`	Prüft, ob eine Liste leer ist.
`^ lengthOf(l : list<Type>) : integer`	Gibt die Anzahl der Listenelemente zurück
`append(l : list<Type>, element : Type) : list<Type>`	Hängt ein Element an einer Liste an
`insert(l : list<Type>, element : Type, pos : integer) : list<Type>`	Fügt ein Element in eine Liste an der gegebenen Position ein.

Ein Beispiel einer Shopping-Liste:

```
blub.mule
1  program blub
2  import IO
3  import Strings
4  import Lists  // Lists library contains a set of operations to be used with lists
5
6  main
7    variable shoppingList : list<string>
8    shoppingList := ["eggs", "milk", "apples", "bread"]
9
10   IO.writeString(shoppingList[0] & "\n")     // eggs
11   IO.writeString(shoppingList[2] & "\n")     // apples
12 endmain
```

Abbildung 8: Quellcode einer Shopping-Liste

5.7 Part 07 – Der Hangman-Galgen

Das Spiel benötigt 7 Hangman-Bilder für die Ausgabe, wenn der Spieler ein Zeichen falsch geraten hat. Die Bilder werden in einer Methode als Liste implementiert. Die entsprechende Eigenschaft wird so deklariert: `private attribute aPics : list<string>`. In der Listenvariable werden alle 7 Bilder als Strings definiert. Zur Erinnerung eine Liste mit einigen Strings:

`lst := ["Erbsen", "Bohnen", "Tomaten", "Möhren"].`

Aufgabe: Schreiben Sie eine Methode `setPics()`, die alle 7 Bilder als Liste

`aPics` implementiert. Die Methode soll noch eine zusätzlich Eigenschaft `private attribute aPicsLen : integer` zur Verfügung stellen, die die Anzahl der Listenelemente zur Verfügung stellt. Schreiben Sie weiterhin eine Methode `showPic(parameter i : integer)`, die bei Aufruf ein Hangman-Bild auf dem Monitor ausgibt. Der Parameter ist der Index den Listenelements, also die Auswahl des Bildes.

Zeitrichtwert: 8 Stunden

5.8 Part 08 – Der Hangman-Galgen

8. Anlagen

8.1 ASCII-Tabelle

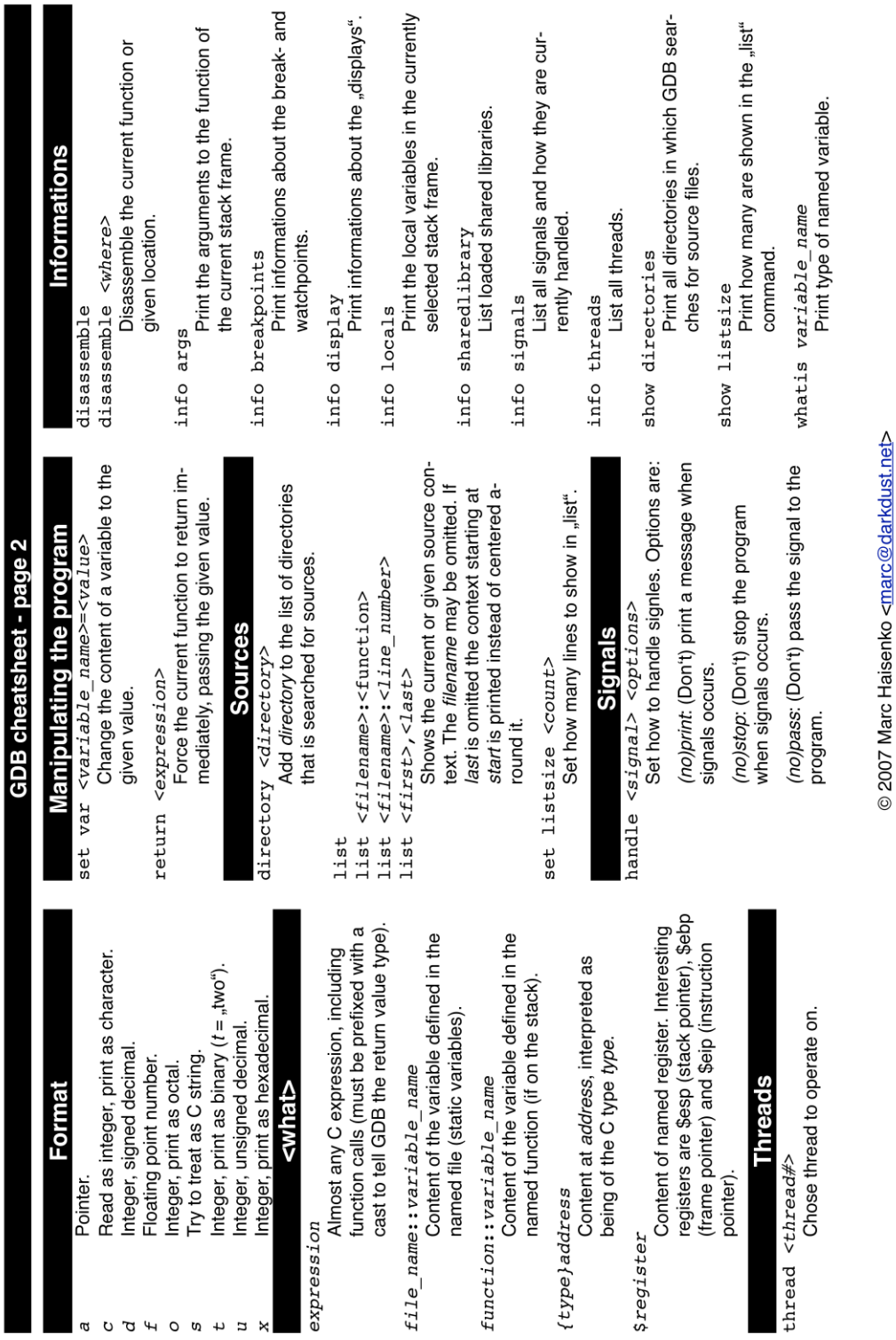
000	NUL	033	!	066	B	099	c	132	ä	165	Ñ	198	ã	231	þ
001	Start Of Header	034	"	067	C	100	d	133	à	166	ª	199	Ã	232	Þ
002	Start Of Text	035	#	068	D	101	e	134	å	167	º	200	╚	233	Ú
003	End Of Text	036	$	069	E	102	f	135	ç	168	¿	201	╔	234	Û
004	End Of Transmission	037	%	070	F	103	g	136	ê	169	®	202	╩	235	Ù
005	Enquiry	038	&	071	G	104	h	137	ë	170	¬	203	╦	236	ý
006	Acknowledge	039		072	H	105	i	138	è	171	½	204	╠	237	Ý
007	Bell	040	(	073	I	106	j	139	ï	172	¼	205	═	238	¯
008	Backspace	041	)	074	J	107	k	140	î	173	¡	206	╬	239	´
009	Horizontal Tab	042	*	075	K	108	l	141	ì	174	«	207	¤	240	-
010	Line Feed	043	+	076	L	109	m	142	Ä	175	»	208	ð	241	±
011	Vertical Tab	044	,	077	M	110	n	143	Å	176	░	209	Ð	242	‗
012	Form Feed	045	-	078	N	111	o	144	É	177	▒	210	Ê	243	¾
013	Carriage Return	046	.	079	O	112	p	145	æ	178	▓	211	Ë	244	¶
014	Shift Out	047	/	080	P	113	q	146	Æ	179	│	212	È	245	§
015	Shift In	048	0	081	Q	114	r	147	ô	180	┤	213	ı	246	÷
016	Delete	049	1	082	R	115	s	148	ö	181	Á	214	Í	247	¸
017	-- frei --	050	2	083	S	116	t	149	ò	182	Â	215	Î	248	°
018	-- frei --	051	3	084	T	117	u	150	û	183	À	216	Ï	249	¨
019	-- frei --	052	4	085	U	118	v	151	ù	184	©	217	┘	250	·
020	-- frei --	053	5	086	V	119	w	152	ÿ	185	╣	218	┌	251	¹
021	Negative Acknowledge	054	6	087	W	120	x	153	Ö	186	║	219	█	252	³
022	Synchronous Idle	055	7	088	X	121	y	154	Ü	187	╗	220	▄	253	²
023	End Of Transmission Block	056	8	089	Y	122	z	155	ø	188	╝	221	¦	254	■
024	Cancel	057	9	090	Z	123	{	156	£	189	¢	222	Ì	255	
025	End Of Medium	058	:	091	[	124	|	157	Ø	190	¥	223	▀		
026	Substitude	059	;	092	\	125	}	158	×	191	┐	224	Ó		
027	Escape	060	<	093	]	126	~	159	ƒ	192	└	225	ß		
028	File Seperator	061	=	094	^	127	⌂	160	á	193	┴	226	Ô		
029	Group Seperator	062	>	095	_	128	Ç	161	í	194	┬	227	Ò		
030	Record Seperator	063	?	096	`	129	ü	162	ó	195	├	228	õ		
031	Unit Seperator	064	@	097	a	130	é	163	ú	196	─	229	Õ		
032		065	A	098	b	131	â	164	ñ	197	┼	230	µ		

Abbildung 9: ASCII-Code Tabelle

8.2 Python Referenzkarte

Abbildung 10: C++ Referenzkarte

8.3



GDB cheatsheet - page 2

Format

- `a` Pointer.
- `c` Read as integer, print as character.
- `d` Integer, signed decimal.
- `f` Floating point number.
- `o` Integer, print as octal.
- `s` Try to treat as C string.
- `t` Integer, print as binary (*t* = „two“).
- `u` Integer, unsigned decimal.
- `x` Integer, print as hexadecimal.

<what>

- `expression` Almost any C expression, including function calls (must be prefixed with a cast to tell GDB the return value type).
- `file_name::variable_name` Content of the variable defined in the named file (static variables).
- `function::variable_name` Content of the variable defined in the named function (if on the stack).
- `{type}address` Content at *address*, interpreted as being of the C type *type*.
- `$register` Content of named register. Interesting registers are $esp (stack pointer), $ebp (frame pointer) and $eip (instruction pointer).

Threads

- `thread <thread#>` Chose thread to operate on.

Manipulating the program

- `set var <variable_name>=<value>` Change the content of a variable to the given value.
- `return <expression>` Force the current function to return immediately, passing the given value.

Sources

- `directory <directory>` Add *directory* to the list of directories that is searched for sources.
- `list`
- `list <filename>:<function>`
- `list <filename>:<line_number>`
- `list <first>,<last>` Shows the current or given source context. The *filename* may be omitted. If *last* is omitted the context starting at *start* is printed instead of centered around it.
- `set listsize <count>` Set how many lines to show in „list“.

Signals

- `handle <signal> <options>` Set how to handle signles. Options are:
  - *(no)print*: (Don't) print a message when signals occurs.
  - *(no)stop*: (Don't) stop the program when signals occurs.
  - *(no)pass*: (Don't) pass the signal to the program.

Informations

- `disassemble`
- `disassemble <where>` Disassemble the current function or given location.
- `info args` Print the arguments to the function of the current stack frame.
- `info breakpoints` Print informations about the break- and watchpoints.
- `info display` Print informations about the „displays“.
- `info locals` Print the local variables in the currently selected stack frame.
- `info sharedlibrary` List loaded shared libraries.
- `info signals` List all signals and how they are currently handled.
- `info threads` List all threads.
- `show directories` Print all directories in which GDB searches for source files.
- `show listsize` Print how many are shown in the „list“ command.
- `whatis variable_name` Print type of named variable.

© 2007 Marc Haisenko <marc@darkdust.net>

Abbildung 11: GDB Referenzkarte